LISTA DE EXERCÍCIOS

UPDATE e ALTER TABLE no MySQL

Conteúdo: criação e população de tabelas, inclusão de colunas e atualização segura de registros

Orientações gerais

Nos exercícios 1 a 6, pratique CREATE TABLE, INSERT, ALTER TABLE e UPDATE.

Nos exercícios 7 a 10, concentre-se em atualizar valores existentes com condições.

- Execute os comandos em um banco de testes; os exemplos abaixo usam tabelas independentes e podem ser recriados para cada exercício.
- Antes de cada UPDATE, rode um SELECT com o mesmo WHERE para verificar exatamente quais linhas serão afetadas.
- Inclua uma condição WHERE adequada. Um UPDATE sem WHERE pode alterar todas as linhas.
- Os exemplos pressupõem uma instalação MySQL com suporte a InnoDB. Em atividades reais, considere transações e cópias de segurança.

Parte I: Enunciados

1. Alunos e e-mails institucionais

Crie a tabela alunos com id (INT, chave primária e auto incremento), nome (VARCHAR(100)), curso (VARCHAR(80)) e semestre (INT).

Cadastre Ana Lima (ADS, 2º semestre), Bruno Alves (ADS, 4º), Carla Souza (Redes de Computadores, 3º) e Diego Reis (Redes de Computadores, 1º).

- Adicione a coluna email_institucional, VARCHAR(120).
- Preencha cada e-mail no padrão nome.sobrenome@faculdade.edu.br, em letras minúsculas e sem espaços.
- Exiba nome, curso e e-mail institucional.

2. Produtos e situação do estoque

Crie produtos com id (INT, chave primária e auto incremento), nome (VARCHAR(100)), categoria (VARCHAR(50)), estoque (INT) e preco (DECIMAL(10,2)).

Cadastre Teclado/Periféricos/12/120, Mouse/Periféricos/4/65, Monitor/Informática/0/890 e Cadeira/Móveis/3/650.

- Adicione situacao_estoque, VARCHAR(20).

- Classifique: estoque 0 = Sem estoque; de 1 a 5 = Baixo; acima de 5 = Disponível.
- Liste nome, estoque e situação.

3. Funcionários e faixa salarial

Crie funcionarios com id (INT, chave primária e auto incremento), nome (VARCHAR(100)), cargo (VARCHAR(80)) e salario (DECIMAL(10,2)).

Cadastre Marina Costa/Analista/4200, Paulo Mendes/Desenvolvedor/6800, Renata Silva/Assistente/2800 e Tiago Rocha/Desenvolvedor/5300.

- Adicione faixa_salarial, VARCHAR(20).
- Classifique: salário menor que 3000 = Inicial; de 3000 até 5000 = Intermediária; acima de 5000 = Avançada.
- Exiba nome, salário e faixa salarial.

4. Livros e classificação do acervo

Crie livros com id (INT, chave primária e auto incremento), titulo (VARCHAR(150)), autor (VARCHAR(100)) e ano_publicacao (INT).

Cadastre Banco de Dados Essencial/L. Pereira/2018; Programação Web/M. Santos/2023; Introdução à Computação/R. Alves/2012; MySQL na Prática/C. Martins/2021.

- Adicione classificacao, VARCHAR(25).
- Classifique: antes de 2015 = Acervo antigo; de 2015 a 2020 = Acervo intermediário; após 2020 = Acervo recente.
- Consulte título, ano e classificação.

5. Clientes e região de atendimento

Crie clientes com id (INT, chave primária e auto incremento), nome (VARCHAR(100)), cidade (VARCHAR(80)) e estado (CHAR(2)).

Cadastre Aline Prado/Campinas/SP; Marcos Nunes/Santos/SP; Beatriz Lima/Curitiba/PR; Eduardo Freitas/Londrina/PR.

- Adicione regiao, VARCHAR(20).
- Preencha Sudeste para clientes de SP e Sul para clientes de PR.
- Mostre nome, cidade, estado e região.

6. Chamados e data de encerramento

Crie chamados com id (INT, chave primária e auto incremento), descricao (VARCHAR(200)), status (VARCHAR(20)) e data_abertura (DATE).

Cadastre: Erro ao acessar o sistema/Aberto/2026-03-02; Instalação de programa/Concluído/2026-03-05; Falha na impressora/Concluído/2026-03-06; Solicitação de acesso/Aberto/2026-03-08.

- Adicione data_encerramento, DATE, aceitando NULL.
- Defina 2026-03-10 para chamados concluídos.
- Mantenha NULL nos chamados ainda abertos.

- Consulte todos os chamados com as duas datas.

7. Reajuste de preços por categoria

Crie produtos com id, nome, categoria e preco (DECIMAL(10,2)).

Cadastre Teclado/Periféricos/100; Mouse/Periféricos/50; Mesa/Móveis/450; Cadeira/Móveis/300.

- Aplique reajuste de 10% somente aos produtos da categoria Periféricos.
- Consulte os preços antes e depois.
- Confirme que os produtos de Móveis não mudaram.

8. Atualização do semestre dos alunos

Crie alunos com id, nome, semestre e situacao.

Cadastre Júlia Ramos/1/Ativo; Pedro Lima/3/Ativo; Sofia Melo/5/Trancado; Caio Dias/7/Ativo.

- Avance em um semestre somente os alunos Ativos.
- Não permita semestre maior que 8.
- Consulte e veja quem foi alterado.

9. Aumento salarial por departamento

Crie equipe com id, nome, departamento e salario (DECIMAL(10,2)).

Cadastre Nádia Freire/Suporte/3000; André Lima/Suporte/3500; Luana Castro/Desenvolvimento/6000; Felipe Moraes/Desenvolvimento/5500.

- Aplique aumento de 5% somente ao departamento Suporte.
- Mostre os salários depois da alteração.
- Confirme que Desenvolvimento permaneceu sem alteração.

10. Cancelamento de pedidos pendentes

Crie pedidos com id, cliente, status e data_pedido.

Cadastre Elisa Martins/Pendente/2026-01-10; Rafael Souza/Pago/2026-02-18; Camila Rocha/Pendente/2026-01-22; Vitor Alves/Pendente/2026-03-03.

- Altere para Cancelado somente pedidos Pendente feitos antes de 2026-02-01.
- Antes do UPDATE, faça SELECT para conferir os pedidos afetados.
- Depois, consulte todos e confirme que os pagos e pendentes mais recentes foram preservados.

Parte II: Resolução dos exercícios

As resoluções são uma possibilidade correta de implementação. Nos exemplos, cada exercício pode ser executado separadamente. Para simplificar a leitura, os INSERTs usam colunas explicitamente nomeadas e os UPDATEs são precedidos por consultas de conferência.

1. Alunos e e-mails institucionais

```
CREATE TABLE alunos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  curso VARCHAR(80) NOT NULL,
  semestre INT NOT NULL
);

INSERT INTO alunos (nome, curso, semestre) VALUES
('Ana Lima', 'Análise e Desenvolvimento de Sistemas', 2),
('Bruno Alves', 'Análise e Desenvolvimento de Sistemas', 4),
('Carla Souza', 'Redes de Computadores', 3),
('Diego Reis', 'Redes de Computadores', 1);

ALTER TABLE alunos
ADD COLUMN email_institucional VARCHAR(120);

-- Conferir quais alunos serão atualizados
SELECT id, nome FROM alunos;

UPDATE alunos
SET email_institucional = CASE nome
  WHEN 'Ana Lima' THEN 'ana.lima@faculdade.edu.br'
  WHEN 'Bruno Alves' THEN 'bruno.alves@faculdade.edu.br'
  WHEN 'Carla Souza' THEN 'carla.souza@faculdade.edu.br'
  WHEN 'Diego Reis' THEN 'diego.reis@faculdade.edu.br'
END
WHERE nome IN ('Ana Lima', 'Bruno Alves', 'Carla Souza', 'Diego Reis');

SELECT nome, curso, email_institucional FROM alunos;
```

O CASE associa um endereço específico a cada nome. O WHERE limita a atualização aos quatro registros do exercício.

2. Produtos e situação do estoque

```
CREATE TABLE produtos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  categoria VARCHAR(50) NOT NULL,
  estoque INT NOT NULL,
  preco DECIMAL(10,2) NOT NULL
);

INSERT INTO produtos (nome, categoria, estoque, preco) VALUES
('Teclado', 'Periféricos', 12, 120.00),
('Mouse', 'Periféricos', 4, 65.00),
('Monitor', 'Informática', 0, 890.00),
('Cadeira', 'Móveis', 3, 650.00);

ALTER TABLE produtos
ADD COLUMN situacao_estoque VARCHAR(20);

SELECT id, nome, estoque FROM produtos;
```

```
UPDATE produtos
SET situacao_estoque = CASE
  WHEN estoque = 0 THEN 'Sem estoque'
  WHEN estoque BETWEEN 1 AND 5 THEN 'Baixo'
  ELSE 'Disponível'
END
WHERE id > 0;

SELECT nome, estoque, situacao_estoque FROM produtos;
```

O CASE avalia as faixas em ordem. A condição id > 0 restringe o comando aos registros válidos da tabela.

3. Funcionários e faixa salarial

```
CREATE TABLE funcionarios (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  cargo VARCHAR(80) NOT NULL,
  salario DECIMAL(10,2) NOT NULL
);

INSERT INTO funcionarios (nome, cargo, salario) VALUES
('Marina Costa', 'Analista', 4200.00),
('Paulo Mendes', 'Desenvolvedor', 6800.00),
('Renata Silva', 'Assistente', 2800.00),
('Tiago Rocha', 'Desenvolvedor', 5300.00);

ALTER TABLE funcionarios
ADD COLUMN faixa_salarial VARCHAR(20);

SELECT id, nome, salario FROM funcionarios;

UPDATE funcionarios
SET faixa_salarial = CASE
  WHEN salario < 3000 THEN 'Inicial'
  WHEN salario <= 5000 THEN 'Intermediária'
  ELSE 'Avançada'
END
WHERE id > 0;

SELECT nome, salario, faixa_salarial FROM funcionarios;
```

A condição intermediária usa salario <= 5000 depois de excluir salários menores que 3000; portanto inclui de 3000 até 5000.

4. Livros e classificação do acervo

```
CREATE TABLE livros (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(150) NOT NULL,
  autor VARCHAR(100) NOT NULL,
  ano_publicacao INT NOT NULL
);

INSERT INTO livros (titulo, autor, ano_publicacao) VALUES
('Banco de Dados Essencial', 'L. Pereira', 2018),
('Programação Web', 'M. Santos', 2023),
('Introdução à Computação', 'R. Alves', 2012),
('MySQL na Prática', 'C. Martins', 2021);

ALTER TABLE livros
ADD COLUMN classificacao VARCHAR(25);
```

```
SELECT id, titulo, ano_publicacao FROM livros;

UPDATE livros
SET classificacao = CASE
  WHEN ano_publicacao < 2015 THEN 'Acervo antigo'
  WHEN ano_publicacao <= 2020 THEN 'Acervo intermediário'
  ELSE 'Acervo recente'
END
WHERE id > 0;

SELECT titulo, ano_publicacao, classificacao FROM livros;
```

5. Clientes e região de atendimento

```
CREATE TABLE clientes (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  cidade VARCHAR(80) NOT NULL,
  estado CHAR(2) NOT NULL
);

INSERT INTO clientes (nome, cidade, estado) VALUES
('Aline Prado', 'Campinas', 'SP'),
('Marcos Nunes', 'Santos', 'SP'),
('Beatriz Lima', 'Curitiba', 'PR'),
('Eduardo Freitas', 'Londrina', 'PR');

ALTER TABLE clientes
ADD COLUMN regiao VARCHAR(20);

SELECT id, nome, estado FROM clientes;

UPDATE clientes
SET regiao = CASE
  WHEN estado = 'SP' THEN 'Sudeste'
  WHEN estado = 'PR' THEN 'Sul'
END
WHERE estado IN ('SP', 'PR');

SELECT nome, cidade, estado, regiao FROM clientes;
```

O filtro estado IN (...) evita atualizar registros de outros estados caso a tabela venha a receber novos dados.

6. Chamados e data de encerramento

```
CREATE TABLE chamados (
  id INT PRIMARY KEY AUTO_INCREMENT,
  descricao VARCHAR(200) NOT NULL,
  status VARCHAR(20) NOT NULL,
  data_abertura DATE NOT NULL
);

INSERT INTO chamados (descricao, status, data_abertura) VALUES
('Erro ao acessar o sistema', 'Aberto', '2026-03-02'),
('Instalação de programa', 'Concluído', '2026-03-05'),
('Falha na impressora', 'Concluído', '2026-03-06'),
('Solicitação de acesso', 'Aberto', '2026-03-08');

ALTER TABLE chamados
ADD COLUMN data_encerramento DATE NULL;

SELECT id, descricao, status FROM chamados
```

```
WHERE status = 'Concluído';

UPDATE chamados
SET data_encerramento = '2026-03-10'
WHERE status = 'Concluído';

SELECT id, descricao, status, data_abertura, data_encerramento
FROM chamados;
```

Como a coluna foi criada aceitando NULL e não é atualizada para chamados abertos, estes permanecem sem data de encerramento.

7. Reajuste de preços por categoria

```
CREATE TABLE produtos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  categoria VARCHAR(50) NOT NULL,
  preco DECIMAL(10,2) NOT NULL
);

INSERT INTO produtos (nome, categoria, preco) VALUES
('Teclado', 'Periféricos', 100.00),
('Mouse', 'Periféricos', 50.00),
('Mesa', 'Móveis', 450.00),
('Cadeira', 'Móveis', 300.00);

SELECT id, nome, categoria, preco FROM produtos
WHERE categoria = 'Periféricos';

UPDATE produtos
SET preco = ROUND(preco * 1.10, 2)
WHERE categoria = 'Periféricos';

SELECT id, nome, categoria, preco FROM produtos
ORDER BY id;
```

Resultado esperado: Teclado = 110,00; Mouse = 55,00; Mesa = 450,00; Cadeira = 300,00. ROUND(..., 2) mantém duas casas decimais.

8. Atualização do semestre dos alunos

```
CREATE TABLE alunos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  semestre INT NOT NULL,
  situacao VARCHAR(20) NOT NULL
);

INSERT INTO alunos (nome, semestre, situacao) VALUES
('Júlia Ramos', 1, 'Ativo'),
('Pedro Lima', 3, 'Ativo'),
('Sofia Melo', 5, 'Trancado'),
('Caio Dias', 7, 'Ativo');

SELECT id, nome, semestre, situacao FROM alunos
WHERE situacao = 'Ativo' AND semestre < 8;

UPDATE alunos
SET semestre = semestre + 1
WHERE situacao = 'Ativo'
  AND semestre < 8;
```

```
SELECT id, nome, semestre, situacao FROM alunos
ORDER BY id;
```

Júlia passa ao 2º semestre, Pedro ao 4º e Caio ao 8º. Sofia permanece no 5º por estar com situação Trancado. A condição semestre < 8 evita ultrapassar o limite.

9. Aumento salarial por departamento

```
CREATE TABLE equipe (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  departamento VARCHAR(50) NOT NULL,
  salario DECIMAL(10,2) NOT NULL
);

INSERT INTO equipe (nome, departamento, salario) VALUES
('Nádia Freire', 'Suporte', 3000.00),
('André Lima', 'Suporte', 3500.00),
('Luana Castro', 'Desenvolvimento', 6000.00),
('Felipe Moraes', 'Desenvolvimento', 5500.00);

SELECT id, nome, departamento, salario FROM equipe
WHERE departamento = 'Suporte';

UPDATE equipe
SET salario = ROUND(salario * 1.05, 2)
WHERE departamento = 'Suporte';

SELECT id, nome, departamento, salario FROM equipe
ORDER BY id;
```

Resultado esperado: Nádia = 3150,00; André = 3675,00. Luana (6000,00) e Felipe (5500,00) permanecem inalterados.

10. Cancelamento de pedidos pendentes

```
CREATE TABLE pedidos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  cliente VARCHAR(100) NOT NULL,
  status VARCHAR(20) NOT NULL,
  data_pedido DATE NOT NULL
);

INSERT INTO pedidos (cliente, status, data_pedido) VALUES
('Elisa Martins', 'Pendente', '2026-01-10'),
('Rafael Souza', 'Pago', '2026-02-18'),
('Camila Rocha', 'Pendente', '2026-01-22'),
('Vitor Alves', 'Pendente', '2026-03-03');

-- Conferir exatamente os pedidos que serão cancelados
SELECT id, cliente, status, data_pedido
FROM pedidos
WHERE status = 'Pendente'
  AND data_pedido < '2026-02-01';

UPDATE pedidos
SET status = 'Cancelado'
WHERE status = 'Pendente'
  AND data_pedido < '2026-02-01';

SELECT id, cliente, status, data_pedido
FROM pedidos
ORDER BY id;
```

Resultado esperado: os pedidos de Elisa e Camila ficam Cancelado. O pedido Pago de Rafael e o pedido Pendente de Vitor permanecem inalterados.

Síntese

- ALTER TABLE acrescenta ou modifica a estrutura de uma tabela; neste material, foi usado para incluir colunas.
- UPDATE modifica valores existentes. O WHERE deve delimitar os registros desejados.
- CASE permite atribuir valores diferentes conforme condições; é útil para preencher uma nova coluna em lote.
- SELECT antes do UPDATE ajuda a prever o efeito da alteração e reduz o risco de modificar dados indevidos.

Dica de laboratório: em operações importantes, use uma transação (START TRANSACTION, confira o resultado e finalize com COMMIT; se houver erro, use ROLLBACK). A disponibilidade e o comportamento transacional dependem do mecanismo de armazenamento da tabela.